COMPUTER GRAPHICS

Assignment 1 – Answers (Simple Notes)

Q1. Hardcopy Technologies in Computer Graphics

Hardcopy means getting a permanent copy of the computer output (text or pictures) on paper, film or other material. The device used for this is called a **hardcopy device**.

- Output can be carried, shared and stored without a computer.
- Two main hardcopy devices: **Printers** and **Plotters**.

A. Printers

- **Impact printers** – a head strikes an inked ribbon against paper (e.g. dot-matrix, daisy-wheel, line printer).
- **Non-impact printers** – no striking; they use ink spray, laser or heat (e.g. inkjet, laser, thermal).

Printer	How it works	Advantage	Disadvantage
Dot-matrix	Pins in the print head hit an inked ribbon and make dots.	Cheap; can print carbon copies.	Noisy; low quality.
Inkjet	Tiny nozzles spray drops of ink on paper.	Good colour; low price.	Ink is costly; slow; can smudge.
Laser	Laser draws image on a charged drum; toner sticks and is heated onto paper.	Very fast; sharp text.	Colour models are costly.
Thermal	Heated pins darken special heat-sensitive paper.	Silent; small size.	Needs special paper; fades.

B. Plotters

A **plotter** draws high-quality **line drawings** (maps, graphs, engineering and CAD drawings) on large sheets using pens or ink.

- **Drum plotter** – paper moves back and forth on a drum while the pen moves sideways.
- **Flatbed plotter** – paper is fixed on a flat table; the pen moves in both X and Y directions.
- **Inkjet plotter** – ink nozzles print colour drawings and posters on wide paper.
- **Electrostatic plotter** – charged dots attract toner; very fast.

Printer vs Plotter

Printer	Plotter
Prints text and raster (dot) images.	Draws vector (line) drawings.
Normally small paper sizes (A4, A3).	Large paper sizes (A0, A1, etc.).
Lines are made of dots.	Lines are smooth and continuous.
Cheap and common.	Costly; used by engineers and architects.

Important characteristics

- Resolution (dots per inch – dpi), speed (pages per minute), colour ability, paper size and cost.

Q2. Display Technologies

Display technology is the technology used to show computer output (text, images, video) on a screen. The screen device is called a **monitor** or display.

1. CRT (Cathode Ray Tube)

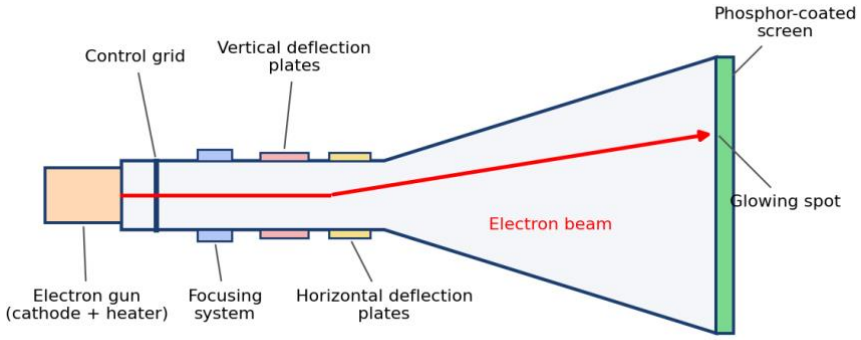


Fig. 1: Cathode Ray Tube (CRT)

- The **electron gun** (heated cathode) produces a beam of electrons.
- The **control grid** controls the intensity (brightness) of the beam.
- The **focusing system** makes the beam narrow and sharp.
- **Deflection plates** move the beam left-right and up-down across the screen.
- When the beam hits the **phosphor** coating, a tiny spot glows. Glow fades quickly, so the picture is redrawn again and again (**refresh**).
- Colour CRT uses three beams (R, G, B) and a **shadow mask** with red, green and blue phosphor dots.

Characteristics: resolution, refresh rate (60 Hz or more), persistence (how long phosphor glows), aspect ratio (4:3), dot pitch (gap between dots).

- Good: cheap, good colour and brightness. Bad: heavy, bulky, uses more power.

2. LCD (Liquid Crystal Display)

- A layer of **liquid crystals** lies between two **polarising** glass sheets.
- A **backlight** at the back gives white light.
- Applying voltage changes the twist of crystals, so they **allow or block** light.
- **RGB colour filters** in front of each pixel make the colours.
- Good: thin, light, low power, no flicker. Bad: limited viewing angle, backlight cannot give perfect black.

3. LED Displays

- **LED-backlit LCD** – uses LEDs as the backlight instead of lamps; thinner, brighter and saves power.
- **OLED** (Organic LED) – each pixel makes its **own light**, so no backlight is needed; true black, wide viewing angle, can be flexible. Costly and may suffer burn-in.

4. Other Display Technologies

- **Plasma** – tiny gas cells glow and excite phosphors; big screens, good colour, but heavy and high power.
- **E-paper (E-ink)** – reflects light like paper; very low power; used in e-readers.
- **Projection (DLP/LCD)** – a projector throws the image on a large screen.

Comparison

Feature	CRT	LCD	LED / OLED	Plasma
Thickness	Very thick	Thin	Very thin	Medium
Power use	High	Low	Low	High
Weight	Heavy	Light	Very light	Heavy
Viewing angle	Wide	Limited	Wide (OLED)	Wide
Cost	Low	Medium	High	High

Q3. Raster-Scan Display System

In a **raster-scan display**, the screen is made of a grid of tiny dots called **pixels**. The electron beam sweeps the screen **row by row (scan line by scan line)** from top to bottom, and the picture is stored as pixel values in memory.

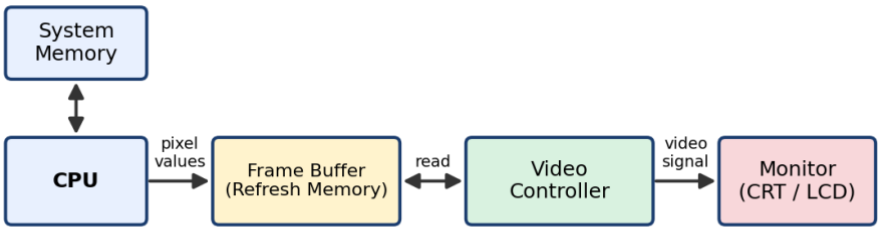


Fig. 2: Raster-scan display system

Role of Frame Buffer (Refresh Buffer)

- It is a memory area that stores the **colour/intensity value of every pixel** on the screen.
- The CPU writes the picture into it; the video controller reads it again and again to refresh the screen.
- **Size = Width × Height × Bits per pixel**. Example: 1024 × 768 at 8 bits = 786,432 bytes = **768 KB**; at 24 bits (true colour) = **2.25 MB**.

Scanning process

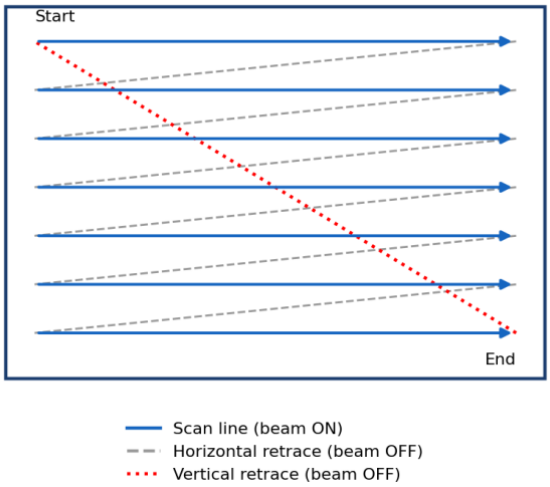


Fig. 3: Scanning process of a raster display

1. The CPU stores pixel values in the frame buffer.
2. The video controller reads the frame buffer line by line.
3. The values are turned into signals that control beam brightness.
4. The beam moves **left → right** along a scan line, then jumps to the next line (**horizontal retrace**, beam off).
5. After the last line, the beam returns to the top-left corner (**vertical retrace**, beam off).
6. This repeats 60 or more times every second (**refresh rate**).

Interlaced vs Non-interlaced

- **Non-interlaced** – all lines are drawn in order in one pass.
- **Interlaced** – odd lines first, then even lines; reduces flicker at low refresh rates (used in TV).

Advantages and Disadvantages

- Advantages: realistic images, many colours, shading and filled areas are easy; low cost.
- Disadvantages: lines look jagged (**aliasing**); needs a large frame buffer.

Q4. Video Controller

A **video controller** (display controller or graphics card) is the hardware that **reads the frame buffer and sends the picture to the monitor**.

Functions

- Continuously reads pixel values from the frame buffer.
- **Refreshes** the screen many times per second.
- Produces **timing, synchronisation and deflection signals** (horizontal and vertical).
- Converts digital pixel values into signals for the monitor (DAC).
- Uses a **colour look-up table** to change a pixel number into an actual RGB colour.
- Can do extra work such as scrolling, zooming and double buffering.

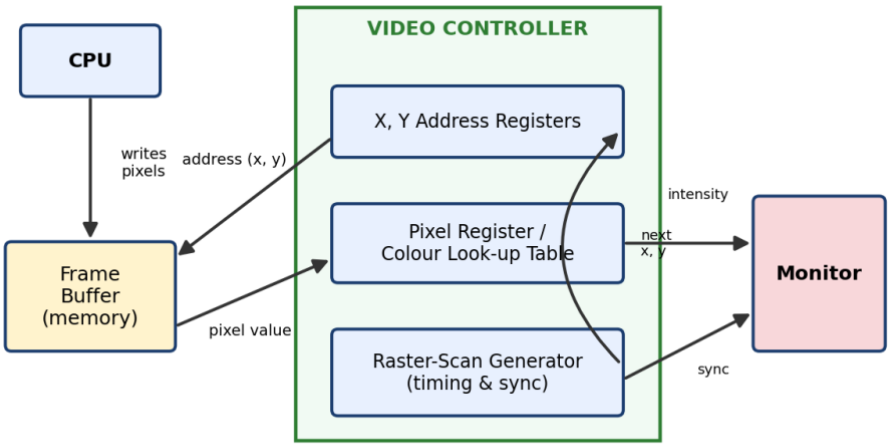


Fig. 4: Video controller

How it controls the display (refresh cycle)

1. Two registers **X** and **Y** hold the current screen position. Set x = 0 and y = top line.
2. Read the pixel value at (x, y) from the frame buffer.
3. Send the intensity of that pixel to the monitor.

4. Increase x by 1 and repeat until the end of the scan line.
5. Reset x = 0 and move y to the next line (horizontal retrace).
6. When all lines are done, reset y to the top (vertical retrace) and start again.

In real hardware, several pixels are fetched together and held in a shift register to make the process faster.

Q5. Random-Scan Display Processor

A **random-scan display** (also called **vector** or **calligraphic** display) draws a picture as a set of **straight lines**. The beam moves directly from one line to the next and only draws where the picture exists.

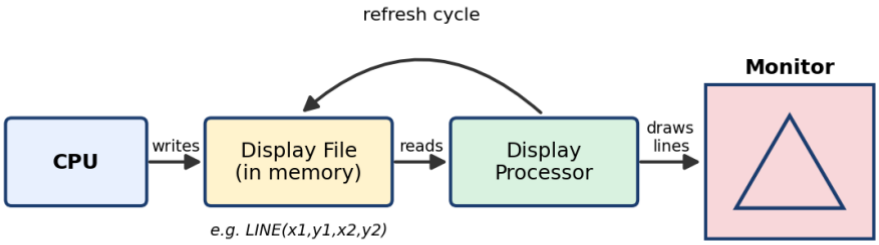


Fig. 5: Random-scan display system

- The picture is stored as drawing commands (line end points) in the **display file / display list**.
- The **display processor** (display controller) reads the display file and draws each line one by one.
- After all lines are drawn, it starts again from the first line (**refresh cycle**, about 30–60 times per second).
- If the picture has too many lines, the refresh becomes slow and the screen **flickers**.

Random-scan vs Raster-scan

Feature	Random-scan	Raster-scan
Picture drawn using	Lines (strokes)	Pixels
Memory needed	Small (display file)	Large (frame buffer)
Line quality	Very smooth, high resolution	Jagged edges (aliasing)
Refresh time	Depends on number of lines	Fixed
Filled areas / colours	Difficult	Easy
Realistic shading	Not possible	Possible
Used for	CAD, line drawings, radar	TV, monitors, games

Q6. Input Devices for Operator Interaction

Input devices let the user give data, positions and commands to the computer. In graphics they are used to **select, position and draw** objects on the screen.

1. Keyboard

- Has keys arranged in a matrix; pressing a key closes a circuit.

- The keyboard controller finds which key was pressed and sends its code (ASCII/scan code) to the computer.
- Used for typing text, numbers and commands; arrow keys move the cursor.

2. Mouse

- **Mechanical mouse** – a rolling ball turns two rollers (X and Y). **Optical mouse** – an LED and sensor track movement over the surface.
- Movement is converted into cursor movement; buttons are used to click and select.

3. Joystick

- A stick on a pivot that can tilt in any direction.
- Two **potentiometers** measure the tilt in X and Y; the cursor moves according to the tilt.
- Used in games, simulators and CAD.

4. Light Pen

- A pen-shaped device with a **photocell** at its tip.
- When pointed at a CRT, it senses the light when the electron beam passes that spot.
- The **time** of detection tells the computer the position, so objects can be picked or drawn.
- Works only on CRT screens.

5. Trackball

- Like an **upside-down mouse** – the ball is rotated with fingers or palm; the body stays still.
- Sensors read the rotation in X and Y. Needs less desk space.

6. Touch Screen

- Detects the **position of a finger touch** directly on the screen.
- **Resistive** – two layers press together; **Capacitive** – finger changes the electric field; **Infrared** – touch breaks a grid of light beams.
- Used in mobiles, ATMs, kiosks and tablets.

Q7. Image Scanner

An **image scanner** is an input device that **converts a printed picture or document into a digital image** that the computer can store and edit.

Working principle

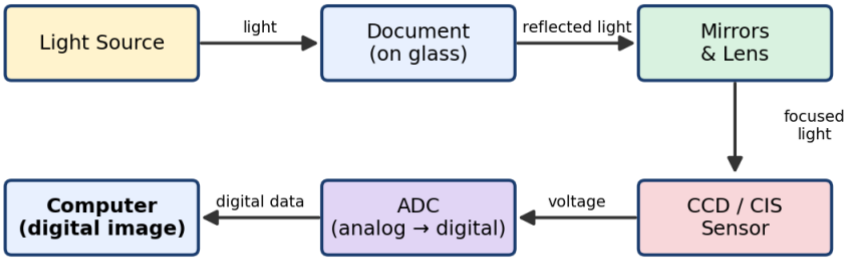


Fig. 6: Working of an image scanner

1. A bright **light source** lights up the document.
2. Light **reflects** from the paper: white areas reflect more, dark areas reflect less.

3. Mirrors and a lens focus the reflected light on a sensor (**CCD** or **CIS**).
4. The sensor changes light into an **electrical signal** (voltage).
5. The **ADC** converts it into digital numbers (pixels).
6. The scan head moves line by line; software joins the lines to make the full image (JPG, PDF, etc.).

Types of scanners

Type	Features	Use
Flatbed	Document placed on a glass plate; scan head moves below.	Most common; photos, books, documents.
Sheet-fed	Paper moves past a fixed sensor.	Fast scanning of many pages.
Handheld	Moved by hand over the document; portable.	Small jobs; less accurate.
Drum	Original is wrapped on a rotating drum; very high quality.	Publishing; costly.
Film / Slide	Special light unit for negatives and slides.	Photography.

Important characteristics

- **Resolution** (dpi) – higher dpi gives more detail.
- **Colour depth** – bits per pixel (24-bit or 48-bit).
- **Scanning speed** – pages per minute.
- **Scan area** – maximum paper size (A4, A3).
- **Dynamic range** – ability to capture dark and light areas.
- **Interface** – USB or Wi-Fi; also OCR software to convert scanned text.

Q8. Cohen–Sutherland Line Clipping Algorithm

Line clipping means removing the part of a line that lies **outside the clipping window**. The Cohen–Sutherland algorithm uses a **4-bit region code** to quickly accept or reject lines.

Region codes

The plane is split into 9 regions. Each point gets a 4-bit code **TBRL**:

- Bit 1 (Top) = 1 if y > ymax
- Bit 2 (Bottom) = 1 if y < ymin
- Bit 3 (Right) = 1 if x > xmax
- Bit 4 (Left) = 1 if x < xmin

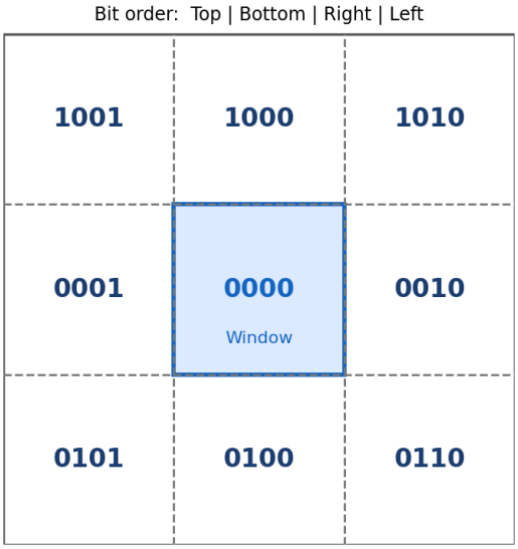


Fig. 7: Region codes (window = 0000)

Algorithm steps

1. Find the region codes of both end points P1 and P2.
2. If both codes are **0000** → line is completely inside → **accept**.
3. If the **AND** of the two codes is **not 0000** → line is completely outside → **reject**.
4. Otherwise the line is partly inside: pick an end point that is outside, and find where the line meets a window edge.
5. Replace the outside point by the intersection point and find its new code.
6. Repeat from step 2 until the line is accepted or rejected.

Intersection formulas

Slope m = (y2 − y1) / (x2 − x1)

```
Left   (x = xmin):  y = y1 + m (xmin − x1)
Right  (x = xmax):  y = y1 + m (xmax − x1)
Top    (y = ymax):  x = x1 + (ymax − y1) / m
Bottom (y = ymin):  x = x1 + (ymin − y1) / m
```

Example

Window: xmin = 20, ymin = 20, xmax = 100, ymax = 80. Line: **P1(10, 30)** to **P2(80, 110)**.

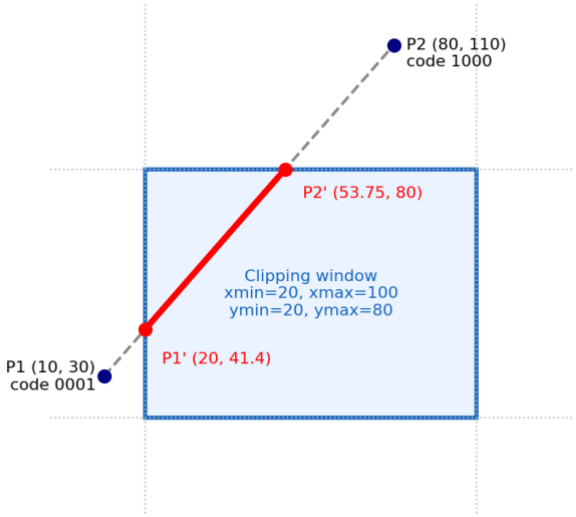


Fig. 8: Clipping example

- Code of P1: x < xmin → **0001**. Code of P2: y > ymax → **1000**.
- AND = 0000 and OR ≠ 0000 → not trivially accepted or rejected, so clip.
- Slope m = (110 − 30) / (80 − 10) = 80/70 = **1.143**.
- Clip P1 with left edge x = 20: y = 30 + 1.143 × (20 − 10) = **41.43** → P1′ = (20, 41.43), code 0000.
- Clip P2 with top edge y = 80: x = 80 + (80 − 110) / 1.143 = **53.75** → P2′ = (53.75, 80), code 0000.
- Both codes are 0000 → accept. **Visible line: (20, 41.43) to (53.75, 80).**

Advantage: very fast accept/reject for most lines. **Disadvantage:** can need many repeats for lines that cross several regions.

Q9. Cyrus–Beck Line Clipping Algorithm

The **Cyrus–Beck** algorithm is a **parametric** line clipping method. Its big advantage is that it works for **any convex polygon** window, not only a rectangle.

Basic equations

Parametric form of the line from P0 to P1:

```
P(t) = P0 + t·D ,   0 ≤ t ≤ 1 ,   where D = P1 − P0
```

For each window edge i, take an **outward normal Ni** and a point **PEi** on that edge. For a point P(t):

- Ni · [P(t) − PEi] < 0 → point is **inside**
- Ni · [P(t) − PEi] = 0 → point is **on the edge**
- Ni · [P(t) − PEi] > 0 → point is **outside**

Put P(t) = P0 + tD and solve for the intersection (value = 0):

```
Ni · (P0 − PEi) + t (Ni · D) = 0
t = Ni · (P0 − PEi) / ( − Ni · D )
```

Meaning of Ni · D

- **Ni · D < 0** → line is **entering** the window at this edge (potential entering, PE).
- **Ni · D > 0** → line is **leaving** the window at this edge (potential leaving, PL).

- **Ni · D = 0** → line is **parallel** to the edge. If Ni · (P0 − PEi) > 0 it is outside → reject; otherwise ignore this edge.

Algorithm steps

1. Find D = P1 − P0.
2. For every edge, find Ni, PEi, Ni · D and Ni · (P0 − PEi).
3. If Ni · D = 0, handle the parallel case as above.
4. Otherwise calculate t. If Ni · D < 0 put it in the **entering** list, else in the **leaving** list.
5. **tE = max(0, all entering t)** and **tL = min(1, all leaving t)**.
6. If **tE > tL** → reject the line. Otherwise the visible part is from **P(tE)** to **P(tL)**.

Example

Window: (0, 0) to (10, 10). Line: **P0(−5, 5)** to **P1(15, 8)**, so D = (20, 3).

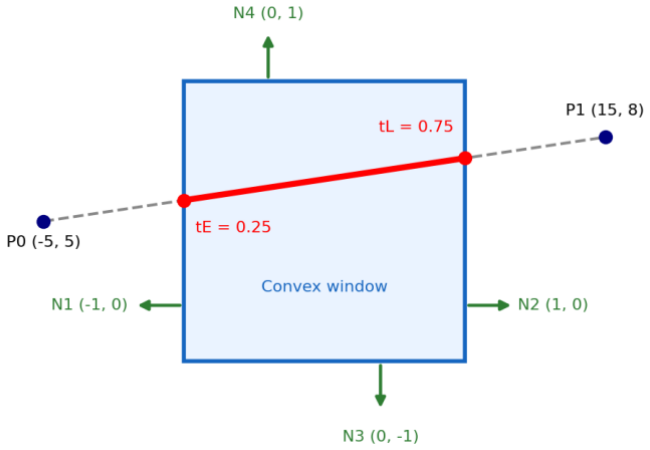


Fig. 9: Cyrus–Beck clipping

Edge	Ni	PEi	Ni · D	t	Type
Left	(−1, 0)	(0, 0)	−20	0.25	Entering
Right	(1, 0)	(10, 0)	20	0.75	Leaving
Bottom	(0, −1)	(0, 0)	−3	−1.67	Entering
Top	(0, 1)	(0, 10)	3	1.67	Leaving

- tE = max(0.25, −1.67, 0) = **0.25**; tL = min(0.75, 1.67, 1) = **0.75**.
- tE < tL → line is visible.
- P(0.25) = (−5 + 5, 5 + 0.75) = **(0, 5.75)**; P(0.75) = (−5 + 15, 5 + 2.25) = **(10, 7.25)**.
- **Clipped line: (0, 5.75) to (10, 7.25).**

Q10. Midpoint Subdivision Line Clipping Algorithm

Midpoint subdivision uses the same region-code tests as Cohen–Sutherland, but finds the window intersection by **repeatedly cutting the line at its midpoint** (like a binary search) instead of solving the line equation.

```
Midpoint  Pm = ( (x1 + x2)/2 ,  (y1 + y2)/2 )
```

Algorithm steps

1. Find region codes of both end points.
2. If both are 0000 → **accept**. If AND ≠ 0000 → **reject**.

3. Otherwise find the midpoint M of the line and its region code.
4. Test each half (P1–M and M–P2): **discard** a half that is completely outside, **keep** a half that is completely inside, and **divide again** if it is partly inside.
5. Repeat until the window edge is found (segment is about one pixel long). The points found are the ends of the visible line.

Example

Window: (0, 0) to (8, 8). Line: **P1(−4, 2)** to **P2(12, 6)**.

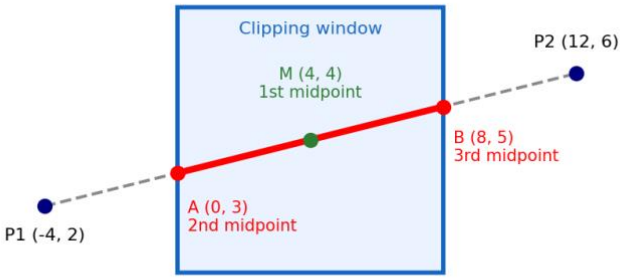


Fig. 10: Midpoint subdivision

Step	Segment	Midpoint	Result
1	P1(−4, 2) – P2(12, 6)	M(4, 4)	Inside window, so both halves are checked.
2	P1(−4, 2) – M(4, 4)	A(0, 3)	On the left edge → visible end found: (0, 3).
3	M(4, 4) – P2(12, 6)	B(8, 5)	On the right edge → visible end found: (8, 5).

Visible line: A(0, 3) to B(8, 5). (Here the midpoints landed exactly on the edges; normally we keep dividing until the part is smaller than a pixel.)

Comparison with Cohen–Sutherland

Feature	Cohen–Sutherland	Midpoint Subdivision
Finding intersection	Solves the line equation (slope, multiply, divide).	Repeated midpoints (only add and divide by 2).
Calculation	Needs floating-point arithmetic.	Simple integer / shift operations.
Accuracy	Exact intersection.	Accurate up to one pixel.
Number of steps	Few (at most 4 clips).	More (about $\log_2$ of line length).
Best suited for	Software.	Hardware implementation.
Region codes	Used.	Used.